House Prices using Backward Elimination

In [23]:
```
#importing all the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

%matplotlib inline

#importing dataset using panda
dataset = pd.read_csv(r"C:\Users\LIKHITH\OneDrive\Desktop\Data science\Data_files\H
#to see what my dataset is comprised of
dataset.head()
```

Out[23]:

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	flo
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	

5 rows × 21 columns

In [24]:
```
#checking if any value is missing
print(dataset.isnull().any())
```

```
id               False
date             False
price            False
bedrooms         False
bathrooms        False
sqft_living      False
sqft_lot         False
floors           False
waterfront       False
view             False
condition        False
grade            False
sqft_above       False
sqft_basement    False
yr_built         False
yr_renovated     False
zipcode          False
lat              False
long             False
sqft_living15    False
sqft_lot15       False
dtype: bool
```

In [25]:
```
#checking for categorical data
print(dataset.dtypes)
```

```
id                int64
date             object
price           float64
bedrooms          int64
bathrooms       float64
sqft_living       int64
sqft_lot          int64
floors          float64
waterfront        int64
view              int64
condition         int64
grade             int64
sqft_above        int64
sqft_basement     int64
yr_built          int64
yr_renovated      int64
zipcode           int64
lat             float64
long            float64
sqft_living15     int64
sqft_lot15        int64
dtype: object
```

In [26]:
```
#dropping the id and date column
dataset = dataset.drop(['id','date'], axis = 1)
```

In [27]:
```
dataset
```

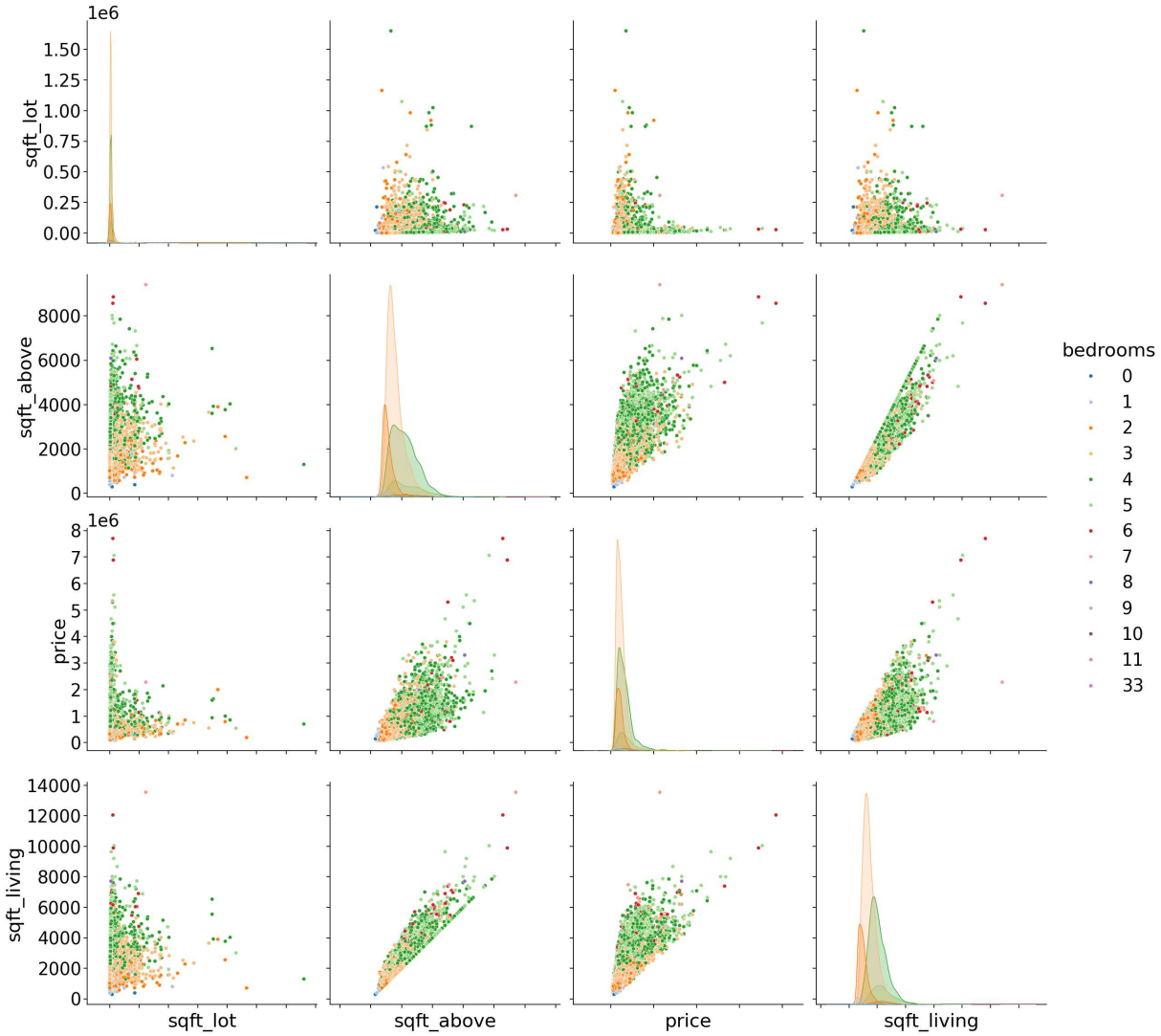
Out[27]:

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	cor
0	221900.0	3	1.00	1180	5650	1.0	0	0	
1	538000.0	3	2.25	2570	7242	2.0	0	0	
2	180000.0	2	1.00	770	10000	1.0	0	0	
3	604000.0	4	3.00	1960	5000	1.0	0	0	
4	510000.0	3	2.00	1680	8080	1.0	0	0	
...	...	...	...	...	...	...	...	...	
21608	360000.0	3	2.50	1530	1131	3.0	0	0	
21609	400000.0	4	2.50	2310	5813	2.0	0	0	
21610	402101.0	2	0.75	1020	1350	2.0	0	0	
21611	400000.0	3	2.50	1600	2388	2.0	0	0	
21612	325000.0	2	0.75	1020	1076	2.0	0	0	

21613 rows × 19 columns

In [28]:

```
# understanding the distribution with seaborn
with sns.plotting_context("notebook", font_scale=2.5):
    g = sns.pairplot(
        dataset[['sqft_lot','sqft_above','price','sqft_living','bedrooms']],
        hue='bedrooms',
        palette='tab20',
        height=6
    )
g.set(xticklabels=[])
plt.show()
```



In [13]:
```
# separating independent and dependent variable
X = dataset.iloc[:,1:].values
y = dataset.iloc[:,0].values

# splitting dataset into training and testing dataset
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=1/3, random_state=0
)
```

In [15]:
```
X
```

Out[15]:
```
array([[ 3.00000e+00,  1.00000e+00,  1.18000e+03, ..., -1.22257e+02,
         1.34000e+03,  5.65000e+03],
       [ 3.00000e+00,  2.25000e+00,  2.57000e+03, ..., -1.22319e+02,
         1.69000e+03,  7.63900e+03],
       [ 2.00000e+00,  1.00000e+00,  7.70000e+02, ..., -1.22233e+02,
         2.72000e+03,  8.06200e+03],
       ...,
       [ 2.00000e+00,  7.50000e-01,  1.02000e+03, ..., -1.22299e+02,
         1.02000e+03,  2.00700e+03],
       [ 3.00000e+00,  2.50000e+00,  1.60000e+03, ..., -1.22069e+02,
         1.41000e+03,  1.28700e+03],
       [ 2.00000e+00,  7.50000e-01,  1.02000e+03, ..., -1.22299e+02,
         1.02000e+03,  1.35700e+03]])
```

In [16]:
```
y
```

Out[16]:
```
array([221900., 538000., 180000., ..., 402101., 400000., 325000.])
```

In [17]:
```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)

# Predicting the Test set results
y_pred = regressor.predict(X_test)
```

In [18]:
```
y_pred
```

Out[18]:
```
array([ 386540.99847813, 1516969.01534083,  538662.72575241, ...,
        526000.75505724,  313924.6366331 ,  400525.67314564])
```

In [19]:
```
#Backward Elimination
import statsmodels.api as sm
def backwardElimination(x, SL):
    numVars = len(x[0])
    temp = np.zeros((21613,19)).astype(int)
    for i in range(0, numVars):
        regressor_OLS = sm.OLS(y, x).fit()
        maxVar = max(regressor_OLS.pvalues).astype(float)
        adjR_before = regressor_OLS.rsquared_adj.astype(float)
        if maxVar > SL:
            for j in range(0, numVars - i):
                if (regressor_OLS.pvalues[j].astype(float) == maxVar):
                    temp[:,j] = x[:, j]
                    x = np.delete(x, j, 1)
                    tmp_regressor = sm.OLS(y, x).fit()
                    adjR_after = tmp_regressor.rsquared_adj.astype(float)
                    if (adjR_before >= adjR_after):
                        x_rollback = np.hstack((x, temp[:,[0,j]]))
                        x_rollback = np.delete(x_rollback, j, 1)
                        print (regressor_OLS.summary())
                        return x_rollback
                    else:
                        continue
    regressor_OLS.summary()
    return x
```

```
SL = 0.05
X_opt = X[:, [0, 1, 2, 3, 4, 5,6,7,8,9,10,11,12,13,14,15,16,17]]
X_Modeled = backwardElimination(X_opt, SL)
```

OLS Regression Results

Dep. Variable:	y	R-squared (uncentered):	0.905
Model:	OLS	Adj. R-squared (uncentered):	0.905
Method:	Least Squares	F-statistic:	1.211e+04
Date:	Tue, 06 Jan 2026	Prob (F-statistic):	0.00
Time:	13:01:53	Log-Likelihood:	-2.9461e+05
No. Observations:	21613	AIC:	5.892e+05
Df Residuals:	21596	BIC:	5.894e+05
Df Model:	17		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
x1	-3.551e+04	1888.716	-18.802	0.000	-3.92e+04	-3.18e+04
x2	4.105e+04	3253.759	12.618	0.000	3.47e+04	4.74e+04
x3	110.2642	2.268	48.607	0.000	105.818	114.711
x4	0.1334	0.048	2.786	0.005	0.040	0.227
x5	5261.5471	3541.347	1.486	0.137	-1679.755	1.22e+04
x6	5.833e+05	1.74e+04	33.598	0.000	5.49e+05	6.17e+05
x7	5.236e+04	2128.298	24.600	0.000	4.82e+04	5.65e+04
x8	2.721e+04	2323.818	11.709	0.000	2.27e+04	3.18e+04
x9	9.548e+04	2145.492	44.503	0.000	9.13e+04	9.97e+04
x10	71.3928	2.238	31.902	0.000	67.006	75.779
x11	38.8714	2.624	14.813	0.000	33.728	44.015
x12	-2561.7953	68.006	-37.670	0.000	-2695.092	-2428.498
x13	20.4187	3.646	5.600	0.000	13.272	27.566
x14	-519.0756	17.826	-29.119	0.000	-554.016	-484.136
x15	6.022e+05	1.07e+04	56.106	0.000	5.81e+05	6.23e+05
x16	-2.179e+05	1.31e+04	-16.683	0.000	-2.44e+05	-1.92e+05
x17	23.0994	3.392	6.811	0.000	16.452	29.747
x18	-0.3761	0.073	-5.137	0.000	-0.520	-0.233

Omnibus:	18403.146	Durbin-Watson:	1.991
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1873534.498
Skew:	3.572	Prob(JB):	0.00
Kurtosis:	48.049	Cond. No.	4.02e+17

Notes:
[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[3] The smallest eigenvalue is 1.36e-21. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

In [ ]: